

Chương 3

LƯU TRỮ VÀ TRUY XUẤT

DỮ LIỆU LỚN

Vũ Ngọc Thanh Sang

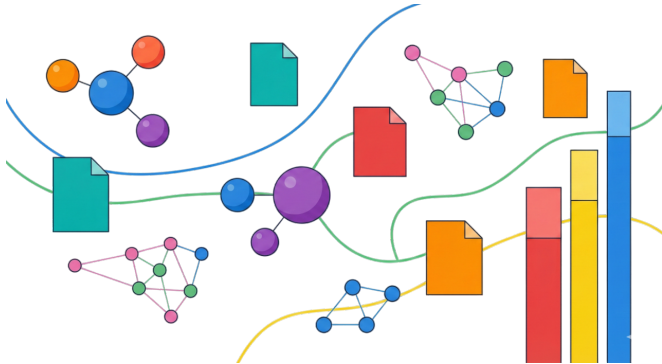
Faculty of Information Technology
Saigon University

01-2026

Mục tiêu bài học

- Hiểu được lý do ra đời và các đặc điểm chính của Cơ sở dữ liệu **NoSQL**.
- Phân loại và phân biệt được 4 mô hình NoSQL chính: **Key-Value, Document, Column-based, và Graph Stores**.
- Nắm vững kiến trúc, mô hình dữ liệu (**Row Key, Column Family, Versioning**) và các thao tác cơ bản (**CRUD**) trong **HBase**.
- Hiểu rõ kiến trúc **Master-less** và cơ chế **Tunable Consistency** của **Cassandra**.
- Phân tích được sự khác biệt giữa **HBase** và **Cassandra** về kiến trúc, hiệu năng và ưu tiên theo định lý **CAP**.
- Áp dụng được các mô hình NoSQL khác nhau vào các trường hợp sử dụng thực tế.

1. Cơ sở dữ liệu NoSQL



1.1 Tổng quan về NoSQL

NoSQL (Not Only SQL)

Là thể hệ cơ sở dữ liệu mới được thiết kế để giải quyết các hạn chế của RDBMS truyền thống trong kỷ nguyên Big Data.

- **Đặc điểm chính:**

- **Không có lược đồ cố định (Schema-less):** Linh hoạt trong việc lưu trữ dữ liệu không cấu trúc hoặc bán cấu trúc.
- **Khả năng mở rộng (Scalability):** Hỗ trợ mở rộng theo chiều ngang trên các cụm máy chủ giá rẻ.
- **Hiệu suất cao:** Tối ưu hóa cho các tác vụ đọc/ghi dữ liệu lớn với độ trễ thấp.
- **Phân loại:** Có 4 loại chính: Key-Value, Document, Column-based, và Graph stores.

1.1.1 Kho lưu trữ Khóa-Giá trị

- **Mô hình dữ liệu:** Lưu trữ dữ liệu dưới dạng cặp <Key, Value>.
- **Đặc điểm:**
 - **Key:** Là định danh duy nhất để truy cập.
 - **Value:** Là một đối tượng "mù" (opaque object) đối với CSDL. Hệ thống chỉ lưu trữ mà không quan tâm nội dung bên trong là gì.
- **Ưu điểm:** Rất nhanh và hiệu quả cho các thao tác đọc/ghi đơn giản dựa trên khóa.
- **Nhược điểm:** Không hỗ trợ truy vấn phức tạp trên nội dung của *Value*.
- **Ví dụ:** Amazon DynamoDB, Voldemort, Redis.

1.1.2 Kho lưu trữ Tài liệu

- **Khái niệm:** Là sự mở rộng của mô hình Key-Value, nhưng *Value* được lưu dưới dạng tài liệu có cấu trúc (JSON, XML, BSON).
- **Sự khác biệt với Key-Value:**
 - Dữ liệu không còn "mù" (opaque).
 - CSDL hiểu cấu trúc bên trong của tài liệu, cho phép truy vấn dựa trên các trường dữ liệu bên trong (ví dụ: tìm tất cả tài liệu có `age > 25`).
- **Đặc điểm:** Mỗi tài liệu có thể có cấu trúc khác nhau (linh hoạt), phù hợp để lưu trữ các đối tượng phức tạp.
- **Ví dụ:** CouchDB, MongoDB.

1.1.3 Kho lưu trữ hướng Cột

- **Nguồn gốc:** Dựa trên kiến trúc Google BigTable.
- **Mô hình dữ liệu:**
 - Dữ liệu được tổ chức thành các **Nhóm cột**.
 - Được coi là một "bảng thưa" khổng lồ, nơi các hàng có thể có các cột khác nhau.
- **Lưu trữ vật lý:** Dữ liệu của cùng một nhóm cột được lưu trữ gần nhau trên đĩa → Tối ưu hóa tốc độ đọc khi chỉ cần truy xuất một số thuộc tính cụ thể.
- **Ví dụ:** Apache HBase, Apache Cassandra, Hypertable.

1.1.4 Kho lưu trữ Đồ thị

- **Mục tiêu:** Thiết kế để lưu trữ và quản lý dữ liệu có mối quan hệ phức tạp (mạng xã hội, hệ thống gợi ý).
- **Thành phần chính:**
 - **Nút:** Đại diện cho thực thể (Người, Địa điểm...).
 - **Cạnh:** Đại diện cho mối quan hệ giữa các nút.
 - **Thuộc tính:** Thuộc tính gắn liền với Node hoặc Edge.
- **Ưu điểm:** Duyệt đồ thị cực nhanh (như duyệt cây), không cần thực hiện các phép JOIN tốn kém như trong RDBMS.
- **Ví dụ:** Neo4j, InfiniteGraph, AllegroGraph.

1.1.5 So sánh các CSDL NoSQL

Loại NoSQL	Ưu điểm nổi bật	Trường hợp sử dụng
Column-based	Tốc độ truy xuất thuộc tính nhanh. Tốt cho phân tích (Analytics).	Kho dữ liệu, Business Intelligence, Big Data Analytics.
Key-Value	Đơn giản, tốc độ cao nếu biết trước khóa.	Caching, Session management, Giỏ hàng.
Document	Linh hoạt, hỗ trợ cấu trúc dữ liệu phức tạp.	Content Management System (CMS), Catalog sản phẩm.
Graph	Xử lý mối quan hệ phức tạp hiệu quả hơn RDBMS.	Mạng xã hội, Hệ thống gợi ý, Phát hiện gian lận.

1.2 Cơ sở dữ liệu NoSQL cho Dữ liệu lớn

1.2.1 Thao tác dữ liệu trong NoSQL

Khác với SQL truyền thống, NoSQL thường tương tác thông qua API lập trình:

- **Thao tác CRUD cơ bản:**
 - **C**reate (Tạo mới)
 - **R**ead (Đọc)
 - **U**ppdate (Cập nhật)
 - **D**eleate (Xóa)
- **Mở rộng SCRUD:** Một số hệ thống bổ sung thêm thao tác Search hoặc Find.
- **Cơ chế thực hiện:** Lập trình viên gọi các hàm/phương thức (API) để đọc ghi đối tượng dữ liệu thay vì chỉ viết câu lệnh truy vấn.

1.2.2 Hạn chế về Ngôn ngữ truy vấn trong NoSQL

Mặc dù một số hệ thống NoSQL cung cấp ngôn ngữ truy vấn cấp cao, nhưng thường có giới hạn:

- **Subset của SQL:** Chỉ cung cấp một tập con các tính năng của SQL, không đầy đủ quyền năng như RDBMS.
- **Vấn đề về phép JOIN:**
 - Nhiều hệ thống NoSQL **không hỗ trợ** phép JOIN trong ngôn ngữ truy vấn.
 - **Giải pháp:** Phép JOIN phải được thực hiện thủ công tại tầng ứng dụng hoặc qua MapReduce.
- **Versioning (Đa phiên bản):** Một số hệ thống hỗ trợ lưu trữ nhiều phiên bản của cùng một mục dữ liệu, phân biệt qua dấu thời gian (timestamp).

Ví dụ 1: MongoDB cho Quản lý Hồ sơ người dùng

Kịch bản: Lưu trữ thông tin người dùng với cấu trúc đa dạng (địa chỉ, sở thích) mà không cần sửa đổi lược đồ bảng.

- **Mô hình dữ liệu (JSON/BSON):**

User Profile Document

```
{
  "_id": "user123",
  "name": "Nguyen Van A",
  "contact": { "email": "vana@email.com", "phone": "0909..." },
  "roles": ["admin", "editor"], // Mảng dữ liệu
  "preferences": {              // Đối tượng lồng nhau
    "theme": "dark",
    "notifications": true
  }
}
```

- **Ứng dụng thực tế:** Hệ thống quản trị nội dung, Catalog sản phẩm thương mại điện tử, nơi mỗi loại sản phẩm có thuộc tính khác nhau.

Ví dụ 2: Neo4j cho Hệ thống Gợi ý Xã hội

Kịch bản: Tìm kiếm "Bạn của bạn" (Friend-of-a-friend) để gợi ý kết bạn hoặc sản phẩm.

- **Mô hình Đồ thị:**

- **Nodes:** User:Alice, User:Bob, Product:Book.
- **Relationships:** (:Alice)-[:FRIEND]->(:Bob), (:Bob)-[:BOUGHT]->(:Book).

- **Truy vấn Cypher (Tìm sách bạn bè đã mua):**

```
MATCH (me:User {name: "Alice"})-[:FRIEND]->(friend)-[:BOUGHT]->(book)
RETURN book.title
```

- **Ưu điểm:** Truy vấn mối quan hệ sâu cực nhanh mà không cần thực hiện nhiều phép JOIN tốn kém như trong SQL.

Ví dụ 3: Amazon DynamoDB (Giỏ hàng trực tuyến)

Kịch bản: Xử lý hàng triệu yêu cầu mua sắm cùng lúc trong ngày Black Friday với độ trễ thấp nhất.

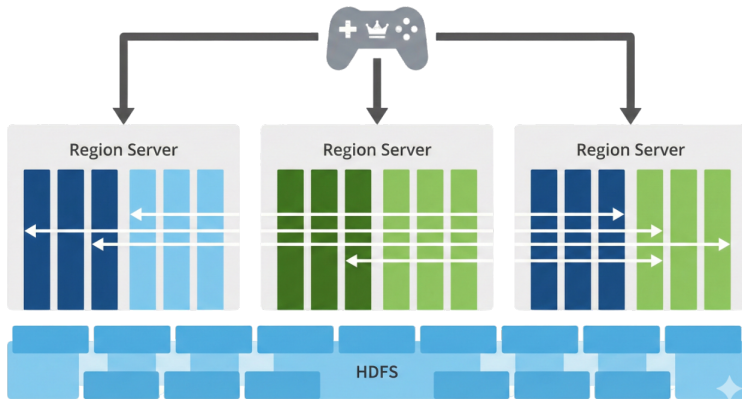
- **Mô hình Key-Value:**

- **Key (SessionID):** session_98765
- **Value (CartBlob):** {items: [{id: 101, qty: 2}, {id: 202, qty: 1}]}

- **Đặc điểm kỹ thuật:**

- **Hiệu suất:** Đọc/ghi cực nhanh nhờ truy cập trực tiếp qua Khóa chính (Primary Key).
- **Khả năng mở rộng:** Tự động phân tán dữ liệu (Sharding) khi lưu lượng truy cập tăng đột biến.
- **Tính nhất quán:** Chấp nhận "Eventual Consistency" (Nhất quán cuối cùng) để đảm bảo giỏ hàng luôn khả dụng (Always-on).

2. HBase



2.1 Tổng quan và Động lực

- **Định nghĩa:** HBase là hệ quản trị cơ sở dữ liệu NoSQL, hướng cột, phi quan hệ và có khả năng mở rộng cực cao, chạy trên nền tảng Hadoop.
- **Cảm hứng thiết kế:** Dựa trên kiến trúc BigTable của Google, sử dụng HDFS để lưu trữ tệp và Google SSTable để lưu trữ dữ liệu nội bộ.
- **Đặc tính cốt lõi:**
 - Cung cấp khả năng đọc/ghi ngẫu nhiên thời gian thực.
 - Hỗ trợ các bảng khổng lồ với hàng tỷ hàng và hàng triệu cột.
 - Phù hợp cho các thao tác không phải MapReduce, cần truy cập nhanh vào các tập con dữ liệu cụ thể.

2.2 Mô hình dữ liệu trong HBase

Dữ liệu được tổ chức theo cấu trúc phân cấp, tự mô tả:

- **Namespace:** Nhóm logic các bảng (tương đương Database trong RDBMS).
- **Table:** Tập hợp các hàng. Tên bảng và Column Family phải được định nghĩa trước.
- **Row Key:**
 - Là một mảng byte, đóng vai trò khóa chính duy nhất.
 - Dữ liệu được sắp xếp theo thứ tự từ điển (lexicographical) dựa trên khóa này.
- **Column Family:** Nhóm các cột liên quan về mặt lưu trữ vật lý (được lưu trong cùng một tệp HDFS).
- **Column Qualifier:** Các thuộc tính cụ thể trong Họ cột, có thể thêm mới linh hoạt mà không cần thay đổi cấu trúc bảng.

2.3 Minh họa: Bảng dữ liệu EMPLOYEE

Row Key	Column Family: Name	Column Family: Details
row1	Fname: Amitabh, Lname: Bacchan	Job: Engineer, Review: Good
row2	Fname: Hema, Mname: S, Lname: Malini	Job: DBA, Supervisor: Prakash
row3	Fname: Prakash, Lname: Padukone	Job: CEO, Salary: 3,100,000

- **Tính linh hoạt:** mỗi hàng có thể có tập cột khác nhau.
- **Truy xuất:** chỉ đọc đúng Column Family cần thiết.

2.4 Khái niệm Cell và Quản lý phiên bản

- **Cell:** Là giao điểm của (Row, Column Family, Column Qualifier). Dữ liệu được lưu dưới dạng mảng byte chưa diễn giải.
- **Versioning:** Mỗi Cell lưu trữ nhiều phiên bản phân biệt bằng **Timestamp** (số nguyên dài biểu thị mili giây).
- **Địa chỉ đầy đủ của một Cell:**
 - {Table, RowID, Column Family, Column Qualifier, Timestamp}
- Người dùng có thể chỉ định số lượng phiên bản muốn giữ lại (ví dụ: chỉ lấy 3 bản ghi mới nhất).

2.5 Kiến trúc hệ thống HBase

HBase phối hợp chặt chẽ với các thành phần trong hệ sinh thái:

- **HMaster (Master Node):** Chịu trách nhiệm giám sát Region Servers, phân chia bảng thành các vùng dữ liệu và gán chúng cho các nút lưu trữ.
- **Region Server:** Lưu trữ và xử lý các dải khóa. Mỗi máy chủ có thể quản lý nhiều Regions.
- **Apache ZooKeeper:** Cung cấp dịch vụ định danh, điều phối và đồng bộ hóa. Nó giúp HMaster biết được trạng thái hoạt động của các Region Servers.
- **HDFS (Lớp lưu trữ):** Đảm bảo tính bền vững của dữ liệu thông qua cơ chế sao chép (Replication) của DataNodes.

2.6 Cơ chế lưu trữ: Regions và Stores

- **Region Partitioning:** Bảng được chia theo dải khóa hàng. Khi một vùng chứa quá nhiều dữ liệu, nó sẽ được **Split** (chia tách).
- **Store:** Trong mỗi Region, mỗi Họ cột (Column Family) được quản lý bởi một Store riêng biệt.
- **MemStore và HFile:** Dữ liệu ghi trước hết vào bộ nhớ (MemStore), sau đó được "flush" xuống tệp tin **SSTable/HFile** trên HDFS.
- **Tính nhất quán:** HBase đảm bảo tính nhất quán cao vì mỗi Region chỉ được phục vụ bởi một Region Server tại một thời điểm.

2.7 Thao tác dữ liệu cơ bản (CRUD API)

HBase cung cấp tập lệnh tối giản để tương tác với dữ liệu:

- **create:** Tạo bảng và định nghĩa Column Families. (*Lưu ý: Không định nghĩa Column Qualifiers lúc này*).
- **put:** Chèn dữ liệu mới hoặc cập nhật phiên bản mới.
 - `put <tablename>, <rowid>, <family>:<qualifier>, <value>`
- **get:** Truy xuất dữ liệu của một hàng cụ thể theo Row Key.
- **scan:** Đọc và duyệt qua tất cả (hoặc một dải) các hàng trong bảng.
- **delete:** Đánh dấu xóa dữ liệu (sử dụng "tombstone" để xóa vật lý sau này).

2.8 Tính chất ACID cấp độ hàng

HBase đáp ứng các tiêu chuẩn ACID tại phạm vi một hàng đơn lẻ:

- **Atomicity (Nguyên tử):** Một phép đột biến trên hàng sẽ thành công hoàn toàn hoặc thất bại hoàn toàn.
- **Consistency (Nhất quán):** Dữ liệu trả về luôn là một hàng hoàn chỉnh, hợp lệ tại thời điểm truy vấn.
- **Isolation (Cô lập):** Cung cấp mức độ cô lập tương đương "read committed" trong DBMS truyền thống.
- **Durability (Bền vững):** Dữ liệu chỉ được coi là thành công khi đã được ghi xuống nhật ký và lưu trữ bền vững trên đĩa.

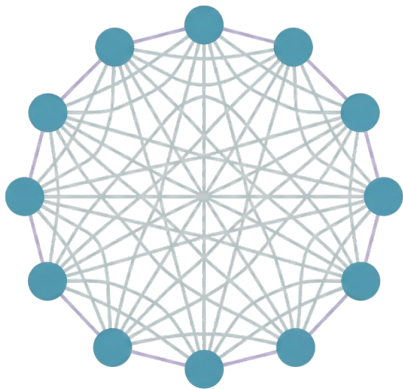
2.9 Khả năng mở rộng và Tích hợp hệ sinh thái

- **Mở rộng ngang:** Thêm Region Servers để tăng dung lượng lưu trữ và khả năng xử lý mà không cần dừng hệ thống.
- **Hỗ trợ MapReduce:** Sử dụng các bảng HBase làm nguồn đầu vào hoặc đích đến cho các tác vụ tính toán song song.
- **Tương tác SQL (Hive/Phoenix):** Cho phép sử dụng các giao diện giống SQL để truy vấn dữ liệu HBase, giúp rút ngắn thời gian phát triển ứng dụng.

2.10 Quy trình ghi dữ liệu trong HBase

1. **Tablet Log:** Giao dịch hợp lệ trước hết được ghi vào nhật ký hệ thống để đảm bảo an toàn.
2. **Memtable:** Dữ liệu sau đó được chèn vào bảng trên bộ nhớ. Lúc này kích thước bộ nhớ sẽ tăng dần.
3. **Flush to Disk:** Khi memtable đạt ngưỡng giới hạn, nó sẽ được chuyển đổi thành tệp **SSTable** và ghi xuống hệ thống tệp HDFS/GFS.
4. **Read View:** Các thao tác đọc sẽ nhìn thấy một "merged view" kết hợp giữa memtable và các SSTable trên đĩa.

3. Cassandra



3.1 Tổng quan về Cassandra

- **Định nghĩa:** Là mô hình dữ liệu phân tán, mã nguồn mở, được thiết kế để xử lý hàng petabyte dữ liệu trên nhiều máy chủ thương mại.
- **Nguồn gốc:** Ban đầu được phát triển bởi Facebook, sau đó trở thành dự án của Apache.
- **Đặc trưng cốt lõi:**
 - Kiến trúc **Master-less** (không nút chính) và **Peer-to-Peer** (ngang hàng).
 - Không có điểm lỗi duy nhất (No single point of failure).
 - Khả năng mở rộng tuyến tính và độ tin cậy cao nhờ sao chép dữ liệu trên nhiều trung tâm dữ liệu.
- **Ngôn ngữ:** Sử dụng Cassandra Query Language (CQL) có cú pháp tương tự SQL.

3.2 Kiến trúc và Tính năng nổi bật

- **Mô hình ngang hàng (P2P):** Tất cả các nút trong cụm đều có vai trò như nhau, giúp đạt thông lượng cao và độ trễ thấp.
- **Phân vùng dữ liệu:** Các hàng được phân vùng vào các bảng. Cột đầu tiên đóng vai trò là khóa phân vùng (Partition Key).
- **Tính sẵn sàng cao:** Dữ liệu được sao chép không đồng bộ giữa các nút ngang hàng. Hệ thống khuyến khích phi chuẩn hóa (denormalization) để tăng tốc độ truy xuất.
- **Mở rộng ngang:** Việc mở rộng chỉ đơn giản là thêm nút mới vào cụm mà không cần dừng hệ thống.

3.3 Quy trình Ghi dữ liệu bền vững

Mỗi thao tác ghi được thực hiện qua các bước để đảm bảo tính bền vững:

1. **Commit Log:** Dữ liệu được ghi ngay lập tức vào nhật ký trên đĩa cứng để phục hồi khi có lỗi phần cứng.
2. **Memtable:** Đồng thời, dữ liệu được lưu vào cấu trúc bộ nhớ đệm (in-memory). Thao tác ghi được coi là thành công khi hoàn tất ở hai bước này.
3. **Flushing:** Khi Memtable đạt ngưỡng giới hạn, dữ liệu được đẩy xuống các tệp **SSTable** (Sorted String Table) trên đĩa thông qua I/O tuần tự.
4. **Compaction:** Các SSTable cũ và không sử dụng sẽ được thu hồi thông qua giai đoạn nén và hợp nhất dữ liệu.

3.4 Khả năng tinh chỉnh mức độ Nhất quán

Cassandra cho phép điều chỉnh sự cân bằng giữa hiệu năng và độ chính xác:

- **ONE:** Chỉ cần một bản sao phản hồi. Tốc độ nhanh nhất nhưng có thể đọc phải dữ liệu cũ (stale data).
- **QUORUM:** Đa số các nút (nút đa số) phải phản hồi. Đảm bảo đọc được dữ liệu mới nhất thông qua cơ chế *Read Repair*.
- **ALL:** Tất cả các nút phải phản hồi. Đảm bảo tính nhất quán tuyệt đối nhưng không có khả năng chịu lỗi (nếu 1 nút hỏng, thao tác sẽ thất bại).

Công thức tính sẵn sàng: $(R + W) > N$

- *W*: số nút ghi tối thiểu, *R*: số nút đọc tối thiểu, *N*: tổng số bản sao.

3.5 Tự phục hồi và Quản lý dung lượng

- **Read Repair:** Trong mức nhất quán thấp (như ONE), nếu dữ liệu trả về bị cũ, hệ thống sẽ tự động cập nhật bản sao mới nhất cho các lần đọc sau.
- **Compaction:** Không gian đĩa từ dữ liệu cũ hoặc lỗi thời sẽ được thu hồi định kỳ thông qua quá trình nén và hợp nhất các SSTable.
- **Thiết kế tối ưu:** Khuyến khích phi chuẩn hóa (denormalization) để xử lý dữ liệu song song, giúp đạt thông lượng cao và độ trễ thấp.

3.7 So sánh Cassandra và HBase (1/3)

Kiến trúc và Phân vùng

Tiêu chí	Apache HBase	Apache Cassandra
Mô hình	Master-Slave (HMaster - Region Server).	Master-less (Peer-to-Peer).
Phân vùng	Theo dải khóa (Range partitioning).	Theo băm hoặc dải trên Partition Key.
Phụ thuộc	Phụ thuộc vào HDFS và ZooKeeper.	Hoạt động độc lập, tự quản lý.
Điểm lỗi	HMaster có thể là điểm lỗi đơn nhất (dù có thể dự phòng).	Không có điểm lỗi đơn nhất (No single point of failure).

3.7 So sánh Cassandra và HBase (2/3)

Hiệu năng và Chịu lỗi

Tiêu chí	Apache HBase	Apache Cassandra
Định lý CAP	Ưu tiên CP (Nhất quán và Chịu lỗi phân vùng).	Ưu tiên AP (Khả dụng và Chịu lỗi phân vùng).
Hiệu năng	Tối ưu cho truy cập ngẫu nhiên, đọc/ghi thời gian thực.	Thông lượng ghi cực cao, độ trễ rất thấp.
Chịu lỗi	Dựa trên cơ chế phục hồi của NameNode và HDFS.	Tự động sao chép giữa các nút ngang hàng và đa trung tâm dữ liệu.
Tính nhất quán	Nhất quán mạnh mẽ (Strong consistency).	Nhất quán cuối cùng (Eventual consistency) hoặc có thể tinh chỉnh.

3.7 So sánh Cassandra và HBase (3/3)

Khi nào chọn Cassandra hay HBase?

- **Chọn HBase khi:**

- Cần tính nhất quán dữ liệu tuyệt đối (ví dụ: giao dịch tài chính).
- Hệ thống đã chạy trên nền tảng Hadoop và tận dụng MapReduce.
- Dữ liệu có cấu trúc cột phức tạp và cần phiên bản hóa (versioning) mạnh.

- **Chọn Cassandra khi:**

- Ưu tiên tính sẵn sàng cao nhất (Always-on).
- Cần tốc độ ghi dữ liệu cực lớn (ví dụ: thu thập dữ liệu IoT, logs).
- Hệ thống trải rộng trên nhiều trung tâm dữ liệu vật lý.

**Thank you
for your
attention!**

